INTEL IMAGE SCENE CLASSIFICATION - PYTORCH AND TENSORFLOW		
Géraud OGOUNCHI	AIMS African Institute for Mathematical Sciences SENEGAL	Deadline
AIMS-SN-2526		**April 2026**
April 16, 2026		2025–2026
	Lecturer: Dr Jordan Felicien Masakuna	

We want to classify natural scene images into 6 categories: **buildings**, **forest**, **glacier**, **mountain**, **sea**, and **street**. The dataset used (`https://www.kaggle.com/datasets/puneet6060/intel-image-classification`) contains around 25,000 images of 150×150 pixels. There are around 14k images in Train, 3k in Test and 7k in Prediction. We perform two kinds of models on this which are **PyTorch** and **TensorFlow**. Full reproducibility is guaranteed by fixing all random seeds to `SEED = 42` across both frameworks at the very start of the pipeline.

My **Live Application URL:** `https://ogb2000-intel-classification-image-last.hf.space`

1 Project Structure

The project is organized into: `models/cnn.py` (model architectures), `models/train.py` (PyTorch Trainer class with early stopping), `utils/prep.py` (data loaders with augmentation), `main.py` (CLI entry point for both frameworks), `app.py` (Flask API), `templates/index.html` (frontend UI), and `Dockerfile` (container configuration).

2 Data Pipeline (`utils/prep.py`)

The entire data pipeline is encapsulated in `utils/prep.py` via the `get_data()` function. It reads images from Kaggle's Intel dataset using `torchvision.datasets.ImageFolder`, which automatically maps sub-folder names to class labels (0=buildings, …, 5=street).

2.1 Preprocessing and Augmentation

Two separate transform pipelines are applied depending on the split:

- **Training transforms:** `RandomResizedCrop(150)`, `RandomHorizontalFlip`, `RandomRotation(10°)`, `ColorJitter` (brightness, contrast, saturation, hue = 0.3/0.3/0.3/0.05), followed by `ToTensor` and ImageNet normalization ($\mu = [0.485, 0.456, 0.406]$, $\sigma = [0.229, 0.224, 0.225]$).
- **Validation transforms(no augmentation):** Only `Resize(150×150)`, `ToTensor`, and the same ImageNet normalization to measure true generalization.

2.2 DataLoader Optimizations

The DataLoaders are built with GPU-aware settings to maximize throughput:

```
DataLoader(dataset, batch_size=32, shuffle=True,
           num_workers=4, pin_memory=use_cuda,
           persistent_workers=True)
```

`pin_memory=True` is activated only when a GPU is detected (`torch.cuda.is_available()`), enabling faster CPU→GPU data transfers. `persistent_workers=True` keeps the 4 worker processes alive across batches, avoiding the overhead of re-spawning them every epoch.

2.3 Train / Validation Split (PyTorch)

The original training folder is further split 80/20 using `random_split` with a seeded generator, creating around 11,200 training samples and around 2,800 validation samples. The official `seg_test` folder serves as the held-out test set.

3. Model Architecture (`models/cnn.py`)

Both models share the same 4-block design philosophy: each block extracts increasingly abstract spatial features by doubling the number of filters, while MaxPooling progressively reduces the spatial resolution (150 → 75 → 37 → 18 → 9). Dropout after each block prevents co-adaptation of neurons.

3.1 PyTorch CNN (`IntelCNN_PyTorch`)

Each convolutional block applies two successive `Conv2d` (kernel 3 × 3, `padding=1` to preserve spatial size), followed by `BatchNorm2d`, `MaxPool2d(2)`, and `Dropout2d`. The dropout rate escalates with depth (0.1, 0.2, 0.3, 0.3) to apply stronger regularization on higher-level features. The **forward pass** is:

$$X \xrightarrow{\text{Block}_1\,(3\to32)} \xrightarrow{\text{Block}_2\,(32\to64)} \xrightarrow{\text{Block}_3\,(64\to128)} \xrightarrow{\text{Block}_4\,(128\to256)} \xrightarrow{\text{AdaptiveAvgPool}} \xrightarrow{\text{FC}(256\to128)} \xrightarrow{\text{Drop}(0.5)} \xrightarrow{\text{FC}(128\to6)}$$

`AdaptiveAvgPool2d(1)` collapses the spatial dimensions to a 256-dimensional vector regardless of input size, making the classifier resolution-agnostic. The output is 6 raw logits, fed into `CrossEntropyLoss` during training.

3.2 TensorFlow/Keras CNN

The Keras model mirrors the PyTorch architecture using the **Functional API**. Key differences: Block 3 uses `Dropout` 0.4 instead of 0.3; Block 4 ends with `GlobalAveragePooling2D` (no MaxPool); the head uses `Dense(256, relu)` with `Dropout(0.6)`, and the output layer applies `softmax` directly.

Table 1: Layer-by-layer architecture comparison between both models.

Component	PyTorch	Keras
Block 1	Conv(3→32)×2, BN, Pool, Drop(0.1)	Conv(3→32)×2, BN, Pool, Drop(0.1)
Block 2	Conv(32→64)×2, BN, Pool, Drop(0.2)	Conv(32→64)×2, BN, Pool, Drop(0.2)
Block 3	Conv(64→128)×2, BN, Pool, Drop(**0.3**)	Conv(64→128)×2, BN, Pool, Drop(**0.4**)
Block 4	Conv(128→256), BN, Pool, Drop(0.3)	Conv(128→256), BN, **GAP**
Head	FC(256→128), Drop(0.5)	Dense(**256**, relu), Drop(**0.6**)
Output	6 logits + CrossEntropy	6 softmax + CategoricalCE

4. Training Strategy (`models/train.py` and `main.py`)

4.1 PyTorch Trainer Class

Training is managed by a dedicated `Trainer` class that encapsulates the optimizer, scheduler, early stopping logic, and evaluation loop. At each epoch, after the training loop finishes, `evaluate()` is called on the validation set using `@torch.no_grad()` to disable gradient computation and save memory.

The **best model saving** logic checks whether validation accuracy improved: if yes, the model state is saved to `geraud_model.pth` and the `no_improve` counter resets to 0; otherwise, `no_improve` increments and training halts once it reaches the patience threshold of 10.

4.2 TensorFlow/Keras Callbacks

The Keras training uses three callbacks chained together:

- **ModelCheckpoint:** saves `geraud_model.keras` whenever `val_accuracy` improves, which correspond to `save_best_only=True` in my code.
- **ReduceLROnPlateau:** halves the learning rate (`factor=0.5`) if `val_loss` does not improve for 3 consecutive epochs, allowing finer convergence near a minimum.
- **EarlyStopping:** halts training after 10 epochs without `val_loss` improvement and automatically restores the best weights (`restore_best_weights=True`).

4.3 Data Augmentation Differences

The two frameworks use slightly different augmentation strategies. PyTorch relies on `ColorJitter` for color distortion. Keras instead uses `layers.RandomZoom(0.15)`, `RandomBrightness` 0.2, and `RandomContrast` 0.2 applied inside the `tf.data` pipeline

Table 2: Hyperparameter settings for each framework.

Hyperparameter	PyTorch	Keras
Optimizer	Adam (lr=1e-3, wd=1e-4)	Adam (lr=5e-4)
LR Scheduler	CosineAnnealingLR ($\eta_{\min}$=1e-6)	ReduceLROnPlateau (factor=0.5, patience=3)
Early Stopping	Patience = 10, monitor val_acc	Patience = 10, restore best weights
Loss	CrossEntropyLoss (logits)	SparseCategoricalCE (softmax)
Batch Size	32	32
Epochs	30	30
Seed	42	42
Val Split	80/20 from train set	Official seg_test folder

6. Results

Model	Accuracy	Loss
PyTorch	77.91% (val)	0.5862 (val)
	75.16% (train)	0.6671 (train)
Keras	**85.63%** (test)	**0.4070**

5. Deployment

The application is deployed as a Flask web service containerized with Docker. It exposes two endpoints: `GET /` renders the interactive HTML frontend, and `POST /predict` accepts an image and returns predictions with confidence scores. The user can switch between PyTorch and Keras backends directly from the UI.